
SmartCliff
CAREER MOBILITY SOLUTIONS
We are on a mission to address the digital skills gap for 10 Million+ young professionals, train and empower them to forge a career path into future tech

Stored Procedure

What is PL/SQL?

- PL/SQL (Procedural Language/SQL) is Oracle's proprietary procedural extension for SQL, used in the Oracle Database.
- It adds programming constructs like **variables, loops, conditionals, exceptions, and packages on top of SQL.**
- **MySQL doesn't have PL/SQL,** the equivalent feature is called **stored programs — which include:**
  - Stored Procedures
  - Functions
  - Triggers
  - Events
- These use **MySQL's procedural extensions to SQL**, but the syntax is **similar, not identical** to PL/SQL.

What is Stored Procedure

- A **stored procedure** in SQL is a **collection of SQL statements saved** and **stored** within the **database**.
- The purpose of the **SQL stored procedure** is to **perform a sequence of operations** on a database, such as **querying, inserting, updating, or deleting data**.
- Unlike regular SQL queries, **executed as separate commands**, **stored procedures encapsulate a set of SQL statements,** making it **easy to reuse the code without having to write SQL commands repeatedly.**

Benefits of Stored Procedure

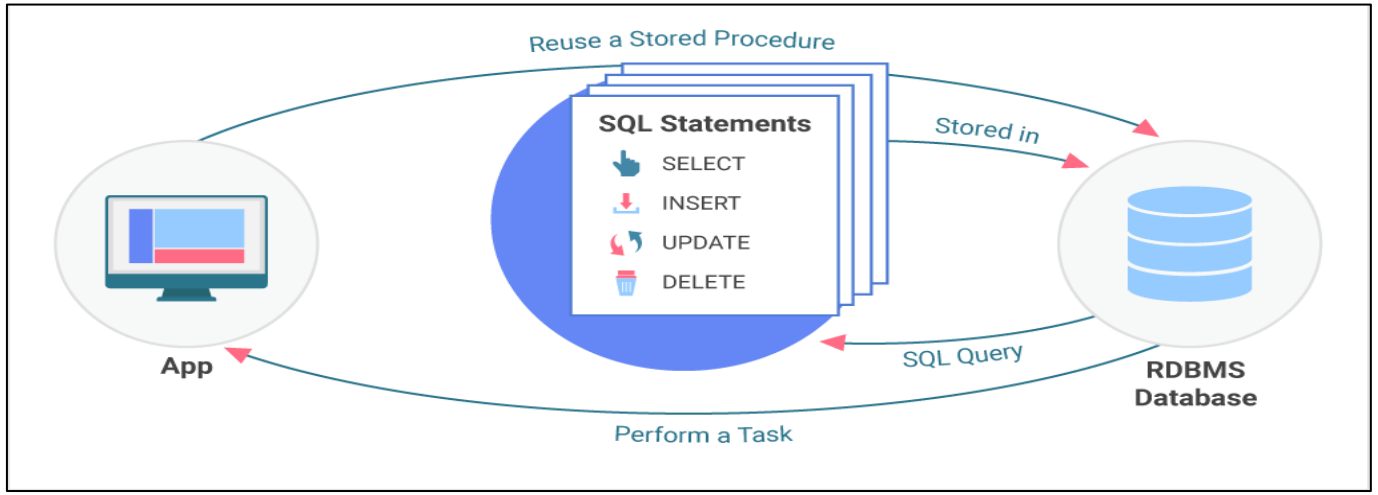
- **Reusability:** Stored procedures allow you to save a set of SQL commands and reuse them multiple times without rewriting the code each time.
- **Execution Optimization:** Stored procedures are pre-compiled and stored on the database server, which can lead to faster execution compared to sending individual queries from a client.
- **Security:** Stored procedures can help control data access and improve security by allowing users to execute the procedure without directly manipulating sensitive data.
- **Modularity:** They promote modular code design, making it easier to organize and maintain database logic.

Benefits of Stored Procedure

- **Parameters and Return Values**: Stored procedures can accept input parameters, allowing them to perform different operations based on the values passed to them and can also return values to the calling application, providing output or status information.

How Stored Procedure Works?

1. **Creation:** A stored procedure is created using SQL commands and stored within the database.
2. **Execution:** It can be executed by calling its name, just like calling a function.
3. **Data Access:** Stored procedures can access and modify data in the database.
4. **Server-Side Execution:** They are executed on the database server, reducing network traffic and improving performance.



Defining a stored procedure

Basic Syntax:

```
DELIMITER //
CREATE PROCEDURE procedure_name(IN param1 datatype, OUT param2 datatype)
BEGIN
    -- SQL statements here
END //

DELIMITER ;
```

Explanation:

- **DELIMITER //:** MySQL normally ends a statement with ;. Since the procedure contains semicolons inside it, we temporarily change the delimiter to //.
- **CREATE PROCEDURE:** Defines the procedure.
- **procedure_name:** The name of the stored procedure.

Defining a stored procedure

- **Parameters:**

✓ **IN:** Input parameter (Params are IN by default and passed into the procedure)

✓ **OUT:** Output parameter (returned from the procedure)

✓ **INOUT:** Can be used for both input and output

✓ **BEGIN ... END:** Marks the body of the procedure.

Note:

You can use various delimiters like DELIMITER $$, DELIMITER //, ;; , @@, or any other combination of characters. The key is to choose a delimiter that does not appear within the SQL statements of the stored procedure.

Defining a stored procedure

Example : Stored procedure that inserts values into multiple tables.

```
-- Create table
CREATE TABLE employees (
    id bigint primary key auto_increment,
    first_name varchar(100),
    last_name varchar(100));

CREATE TABLE birthdays (
    emp_id bigint,
    birthday date,
    constraint foreign key (emp_id) references employees(id)
);
```

Defining a stored procedure

```
--Stored Procedure
delimiter // -- since our stored procedure contains multiple statement separated with ";",
        -- we need to tell the MySQL shell not to try to execute the statement when
        -- it encounters a ";" like it normally would. Instead, it should wait for "//"
CREATE procedure new_employee( -- the name of our procedure is new_employee
   first char(100), -- the first param is called "first" and is a character string
   last char(100),  -- the second param is called "last" and is a character string
   birthday date) -- the third param is called "birthday" and is a date
BEGIN -- since our procedure body has multiple statements, we wrap them in a BEGIN .. END block
   INSERT INTO employees (first_name, last_name) VALUES (first, last); -- insert into employees table
   SET @id = (SELECT last_insert_id()); -- assign the auto-generated ID from the INSERT to a variable
   INSERT INTO birthdays (emp_id, birthday) VALUES (@id, birthday); -- INSERT into the second table
END; -- end of the BEGIN .. END block
//
delimiter ; -- now that we're done defining our procedure, change the delimiter back to ";"
```

Defining a stored procedure

```
--Calling the Stored Procedure
 call new_employee("tim", "sehn", "1980-02-03");
Query OK, 1 row affected (0.02 sec)
```

```
 SELECT * FROM employees;
+----+--------------+-------------+
| id   | first_name | last_name |
+----+--------------+-------------+
|  1    | tim            | sehn        |
+----+-----------+----------------+
1 row in set (0.00 sec)
```

OUT Parameters

- **MySQL stored procedures don't use the return** keyword to **return a value** to the **caller** like other programming languages.
- Instead, you **declare parameters** as **OUT** rather than **IN**, and then they're set in the procedure body. Here's a simple example.

```
--Stored Procedure
delimiter //
CREATE PROCEDURE birthday_count(
    IN bday date,
    OUT count int)
BEGIN
    SET count = (SELECT count(*) FROM birthdays WHERE birthday = bday);
END
//


delimiter ;
```

OUT Parameters

```
-- Calling the Procedure
-- couple other birthdays
CALL new_employee('aaron', 'son', '1985-01-10');
CALL new_employee('brian', 'hendriks', '1985-01-10');
SET @count = 0;
call birthday_count('1985-01-10', @count);
Query OK, 0 rows affected (0.00 sec)

SELECT @count;
+----------+
| @count |
+----------+
|      2       |
+----------+
1 row in set (0.00 sec)
```

Variables

- In SQL, particularly with stored procedures, you can work with different types of variables: **session variables**, **global variables**, and **local variables**. Here's a breakdown of each:

1. Session Variables (@var):

- **Scope: Session variables** are **specific** to **the current database session**. They are **created** and **used within a session**, and their **values are lost when the session ends**.
- **Usage:** They can **store temporary** data such as **results of a query** or **flags within a session**.

Example:

```
-- Declaring and setting a session variable
SET @user_id = 101;
-- Using the session variable
SELECT * FROM users WHERE id = @user_id;
```

In this example, @user_id is a session variable, and it holds the value 101 for the duration of the session.

Variables

2. Global Variables (@@var):

- **Scope: Global variables** are **accessible across all sessions**. They are typically **system-defined variables** that **hold information about the server or environment**. They maintain their **values until the server is restarted.**
- **Usage:** They are often used for **getting server-level information**, such as the **current date, server version, or system variables.**
- **Example:**

```
-- Accessing a global variable
SELECT @@version;  -- Gets the current MySQL version
```

Here, @@version is a global variable that returns the MySQL server version.

Variables

3.Local Variables (inside Stored Procedures or Functions):

- **Scope: Local variables** are **defined** and **used** within a **stored procedure or function**. Their values are **valid only for the duration** of the **procedure or function execution.**
- **Usage:** These are used for **intermediate calculations** or **temporary storage within a specific scope**, and they **don’t persist beyond the procedure's execution**.
- **Example:**

```
--Accessing the Local Variables
DELIMITER $$
CREATE PROCEDURE get_employee_count()
BEGIN
    DECLARE total_employees INT; --
    -- Calculate the total number of employees
    SELECT COUNT(*) INTO total_employees FROM employees;
```

Variables

```
-- Return the total employee count
SELECT total_employees;
END$$
DELIMITER ;
```

- **Stored procedures** can use **session variables (@var), or global variables (@@var), or local variables.**
- **For Example:** We want to create a stored procedure that performs the following tasks:
  - Uses a **session variable** to store today's date.
  - Displays the **MySQL server version** using a **global variable**.
  - Uses a **local variable** to calculate how many employees have a birthday today.
  - Retrieves the list of employees who have their birthdays today.

Variables

```
DELIMITER $$
CREATE PROCEDURE GetEmployeeBirthdayInfo()
BEGIN
    -- Declaring a local variable to store the count of employees with birthdays today
    DECLARE birthday_count INT;

    -- Using a session variable to store today's date
    SET @today = CURDATE();

    -- Displaying the MySQL server version (global variable)
    SELECT CONCAT('MySQL Version: ', @@version) AS mysql_version;
    -- Counting employees with birthdays today
    SELECT COUNT(*) INTO birthday_count
    FROM employees e
    JOIN birthdays b ON e.id = b.emp_id
    WHERE b.birthday = @today;
```

Variables

```
-- Returning the count of employees with birthdays today
   SELECT birthday_count AS 'Employees with Birthday Today';

   -- Selecting the employees with birthdays today
   SELECT e.first_name, e.last_name, b.birthday
   FROM employees e
   JOIN birthdays b ON e.id = b.emp_id
   WHERE b.birthday = @today;
END$$

DELIMITER ;
```

```
--Executing the Stored Procedure:
CALL GetEmployeeBirthdayInfo();
```

Control flow statements

- Just like any other programming language, **MySQL stored procedures support conditional logic** by means of a **set of control flow statements like IF, LOOP**, etc.

IF Statements:

- The **IF statement** is used to **execute one of several possible blocks of code** based on **specific conditions.**
- You can **include multiple ELSEIF clauses** to check **additional conditions,** and **optionally use an ELSE clause** to handle **cases** where none of the previous conditions are met.
- **Each IF or ELSEIF** is followed by a **THEN keyword** to **indicate the start of the corresponding block of statements.**
- **The complete IF statement must be properly closed** with an **END IF.**

Control flow statements

```
CREATE PROCEDURE birthday_message(
   bday date,
   OUT message varchar(100))
BEGIN
   DECLARE count int;
   DECLARE name varchar(100);
   SET count = (SELECT count(*) FROM birthdays WHERE birthday = bday);
   IF count = 0 THEN
      SET message = "Nobody has this birthday";
   ELSEIF count = 1 THEN
      SET name = (SELECT concat(first_name, " ", last_name)
         FROM employees join birthdays
         on emp_id = id
         WHERE birthday = bday);
      SET message = (SELECT concat("It's ", name, "'s birthday"));
```

Control flow statements

```
ELSE
        SET message = "More than one employee has this birthday";
    END IF;
END
-- Calling the Procedure
call birthday_message (now(), @message);
Query OK, 0 rows affected, 1 warning (0.00 sec)

SELECT @message;
+------------------------------+
| @message                     |
+------------------------------+
| Nobody has this birthday |
+------------------------------+
1 row in set (0.00 sec)
```

Control flow statements

```
call birthday_message ('1980-02-03', @message);
Query OK, 0 rows affected (0.01 sec)

SELECT @message;
+---------------------------+
| @message                  |
+---------------------------+
| It's tim sehn's birthday |
+---------------------------+
1 row in set (0.00 sec)
```

Control flow statements

```
call birthday_message ('1985-01-10', @message);


mysql> SELECT @message;
+---------------------------------------+
| @message                              |
+---------------------------------------+
| More than one employee has this birthday |
+---------------------------------------+
1 row in set (0.00 sec)
```

Control flow statements

CASE Statement:

- **CASE statements** are another **way of expressing conditional logic**, when the **same expression is evaluated** for **every logical branch**, **similar** to the **switch statement** found in many programming languages.
- We can implement the **same procedure** above **using case statements** instead. Note that you end a **CASE block with a END CASE statement.**

```
CREATE PROCEDURE birthday_message(
   bday date,
   OUT message varchar(100))
BEGIN
   DECLARE count int;
   DECLARE name varchar(100);
   SET count = (SELECT count(*) FROM birthdays WHERE birthday = bday);
```

Control flow statements

```
CASE count
      WHEN 0 THEN
         SET message = "Nobody has this birthday";
      WHEN 1 THEN
         SET name = (SELECT concat(first_name, " ", last_name)
            FROM employees join birthdays
            on emp_id = id
            WHERE birthday = bday);
         SET message = (SELECT concat("It's ", name, "'s birthday"));
      ELSE
         SET message = "More than one employee has this birthday";
   END CASE;
END
```

- Compared to using IF and ELSEIF, the CASE version makes it clearer that the choice of execution path depends on the value of the count variable.

Control flow statements

Loop Constructs:

- To repeat the **same set of statements more than once**, use a **loop construct**.
- **MySQL** provides **three different loop constructs** to **choose from: WHILE, REPEAT, and LOOP**.
- **For this example,** we'll write a **procedure** that computes the **Nth fibonnaci number** and **assigns it to an out parameter.**

WHILE:

```
-- Stored Procedure
CREATE PROCEDURE fib(n int, out answer int)
BEGIN
   DECLARE i int default 2;
   DECLARE p, q int default 1;
   SET answer = 1;
```

```
WHILE i < n DO
      SET answer = p + q;
      SET p = q;
      SET q = answer;
      SET i = i + 1;
   END WHILE;
END;
```

Control flow statements

```
-- Calling the Procedure
SET @answer = 1; call fib(6, @answer); SELECT @answer;
Query OK, 0 rows affected (0.00 sec)
+------------+
| @answer |
+------------+
|      8 |
+------------+

SET @answer = 1; call fib(7, @answer); SELECT @answer;
Query OK, 0 rows affected (0.00 sec)
+------------+
| @answer |
+------------+
|     13      |
+-----------+
```

Note the use of the **DEFAULT** keyword on the DECLARE statements, which we hadn't used before. **This assigns an initial value to the variable**. Unlike other languages, MySQL integer variables do not default to 0 or any other value, but instead are initialized to NULL by default (which you don't want for calculation).

Control flow statements

REPEAT

- Unlike WHILE, **REPEAT loops check the loop condition at the end of the loop, not the beginning.** So, they always **execute the body of the loop at least once.**

```
CREATE PROCEDURE fib(n int, out answer int)
BEGIN
   DECLARE i int default 1;
   DECLARE p int default 0;
   DECLARE q int default 1;
   SET answer = 1;
```

```
REPEAT
      SET answer = p + q;
      SET p = q;
      SET q = answer;
      SET i = i + 1;
   UNTIL i >= n END REPEAT;
END;
```

Control flow statements

LOOP, ITERATE and LEAVE

- Finally let's look at LOOP. Unlike REPEAT and WHILE, **LOOP has no built-in exit condition**, making it very easy to write an **infinite loop**. You have to **use a label** and **code an explicit LEAVE** statement to **exit the loop**. Here's the same procedure again, with the loop1 label applied:

```
CREATE PROCEDURE fib(n int, out answer int)
BEGIN
   DECLARE i int default 2;
   DECLARE p, q int default 1;
   SET answer = 1;
   loop1: LOOP
      IF i >= n THEN
         LEAVE loop1;
      END IF;
```

```
SET answer = p + q;
      SET p = q;
      SET q = answer;
      SET i = i + 1;
   END LOOP loop1;
END;
```

Note that the loop1 label occurs both before the LOOP as well as at the end of it. The LEAVE statement terminates the loop.

Exception Handling

In **MySQL stored procedures**, the **SIGNAL keyword** is a **way to programmatically raise an error**.

This is useful when you want to:

- Halt execution when a custom validation or logic fails
- Prevent invalid data from being inserted or processed
- Force a rollback of a transaction for maintaining data integrity

Example: Imagine you're writing a stored procedure to insert a new student record into a university database, but you want to prevent inserting students under 17 years old.

Exception Handling

```
DELIMITER $$
CREATE PROCEDURE AddStudent(
    IN name VARCHAR(100),
    IN age INT
)
BEGIN
    IF age < 17 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Student must be at least 17 years old.';
    END IF;

    INSERT INTO students (student_name, student_age)
    VALUES (name, age);
END$$

DELIMITER ;
```

Now, if someone calls AddStudent('John', 15), the **procedure throws an error and stops execution**. No data is inserted.

Quiz

1) In MySQL, what keyword is used to define the start and end of a stored procedure body?

a) BEGIN ... END

b) START ... STOP

c) OPEN ... CLOSE

d) PROCEDURE ... ENDPROC

Answer : Option a)

Quiz

2) What SQL command is used to execute a stored procedure?

a) Execute

b) Run

c) CALL

d) Exec

Answer : Option c)

Quiz

3) What is a Stored Procedure in MySQL?

a) A precompiled SQL program stored in the database

b) A trigger that runs automatically

c) A function to create views

d) A type of table

Answer : Option a)

Quiz

4) Which keyword is used to define a loop block in a MySQL stored procedure?

a) FOR

b) WHILE

c) LOOP

d) DO

Answer : Option c)

Quiz

5) What is the purpose of the SIGNAL keyword in a MySQL stored procedure?

a) To stop the MySQL server

b) To continue to the next statement forcibly

c) To raise an error condition intentionally

d) None of the above

Answer : Option c)


THANK YOU
SmartCliff
Career Mobility Solutions